

Fast & Slow Pointers Revision

8 problems

Fast recall cards for the current fast and slow pointers package. Picture the runner movement, then check the sample and bug magnets before opening the Java file. Exported in expanded card mode.

#01 LinkedListCycle

Easy

Runner collision check Easy [Source](#) [LeetCode](#)**Before reading: what proves a linked list has a loop?**

PROBLEM DESCRIPTION

Given the head of a linked list, return true if any node can be reached again by following next pointers, otherwise return false.

INPUT

head = [3, 2, 0, -4], pos = 1

OUTPUT

true

TIME

$O(n)$

SPACE

$O(1)$

DATA STRUCTURES

Slow pointer, fast pointer, linked list nodes

WHEN TO RECOGNIZE IT

When following next pointers may eventually loop back to an earlier node, and extra visited storage is not needed.

VISUAL HOOK

3 2 0 -4

slow +1

fast +2

cycle -> runners meet

no cycle -> fast hits null

- 1 Move slow by one node and fast by two nodes.
- 2 If fast reaches null, the list ends and has no cycle.
- 3 If slow and fast meet by reference, both runners are inside the same loop.

Imagine two runners on the same track. If the track loops, the fast runner eventually catches the slow runner. If the track ends, fast reaches null.

CORE MOVE

Move slow one node and fast two nodes while fast and fast.next exist. If they ever point to the same node, a cycle exists. If fast reaches the end, there is no cycle.

CODE SKELETON

```
slow = head; fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
    if slow == fast: return true
return false
```

BRUTE FORCE / BASELINE

Store every visited node in a HashSet and return true when a node repeats. It is straightforward but uses $O(n)$ extra space.

ALTERNATE APPROACHES

- Temporarily marking nodes can detect repeats, but it mutates the list and is usually not acceptable.
- Floyd cycle detection is preferred because it keeps $O(1)$ space and does not modify nodes.

BUG MAGNETS

- Compare node references, not node values.
- Check fast and fast.next before moving fast two steps.
- A null head or one-node list without a self-loop should return false.

#02 LinkedListCycleII

Medium

Find loop entry Medium [Source](#) [LeetCode](#)

Before reading: after the runners collide, where should one pointer restart?

PROBLEM DESCRIPTION

Given the head of a linked list, return the exact node where the cycle begins. If the list has no cycle, return null.

INPUT

head = [3, 2, 0, -4], pos = 1

OUTPUT

node with value 2

TIME
 $O(n)$

SPACE
 $O(1)$

DATA STRUCTURES
Slow pointer, fast pointer,
linked list nodes

WHEN TO RECOGNIZE IT

When a linked list may loop and the answer is not just whether a loop exists, but the first node of the loop.

VISUAL HOOK

3 2 0 -4

phase 1: slow +1, fast +2

meet somewhere inside loop

phase 2: head +1, meet +1

second meeting = cycle start

- 1 First run slow and fast until they meet inside the cycle.
- 2 Restart one pointer at head and leave the other at the meeting node.
- 3 Move both one step at a time; their next meeting is the cycle entry.

First catch the fast runner inside the loop. Then restart one runner at head. When both move one step, they meet exactly at the loop door.

CORE MOVE

Use Floyd's first meeting to prove a cycle exists. Then reset one pointer to head and move both pointers one step at a time until they meet at the cycle entry node.

CODE SKELETON

```
slow = head; fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
    if slow == fast: break
if no meeting: return null
fast = head
while slow != fast:
    slow = slow.next
    fast = fast.next
return slow
```

BRUTE FORCE / BASELINE

Put every visited node reference in a HashSet and return the first node seen twice. It is easy to reason about, but costs $O(n)$ extra space.

ALTERNATE APPROACHES

- Temporarily tagging nodes can reveal the repeated node, but mutates the list and is unsafe for interview constraints.
- Floyd's two-phase method is the usual constant-space solution.

BUG MAGNETS

- After the first meeting, reset only one pointer to head; keep the other at the meeting point.
- In phase two, both pointers move one step, not fast by two.
- Return the node reference, not just the node value.

#03 HappyNumber

Easy

Digit-square cycle Easy [Source](#) [LeetCode](#)**Before reading: what proves the number chain will never reach 1?**

PROBLEM DESCRIPTION

Given a positive integer, repeatedly replace it with the sum of the squares of its digits. Return true if the chain reaches 1, and false if it falls into a repeating cycle.

INPUT

$n = 2$

OUTPUT

false

TIME

$O(\log n)$ per transform;
bounded overall for int inputs

SPACE

$O(k)$ with visited set

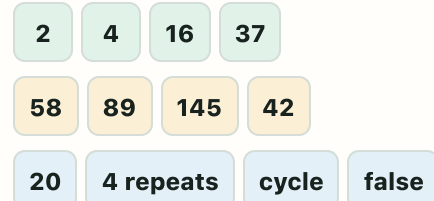
DATA STRUCTURES

HashSet of seen numbers, digit loop

WHEN TO RECOGNIZE IT

When a value repeatedly transforms into another value and the process either reaches a target or repeats forever.

VISUAL HOOK



- 1 Convert the current number into the sum of squared digits.
- 2 If the new value is 1, the number is happy.
- 3 If the new value was seen before, the route is cycling and cannot become happy.

Picture each number as a station in a train route. A happy route reaches station 1. An unhappy route revisits an old station and loops forever.

CORE MOVE

Keep generating the digit-square sum. If the value becomes 1, return true. If a generated value was already seen, return false because the chain is cycling.

CODE SKELETON

```
seen = empty set
add n to seen
while n != 1:
    n = sumOfSquaredDigits(n)
    if n in seen: return false
    add n to seen
return true
```

BRUTE FORCE / BASELINE

Manually simulate for a fixed number of iterations and hope it stops. That can work for small examples but has no clean correctness reason.

ALTERNATE APPROACHES

- Use Floyd's slow/fast pointers on the generated number sequence to detect the cycle in $O(1)$ extra space.
- For base-10 integers, the sequence quickly falls into a small bounded range, so either a HashSet or Floyd is fast in practice.

BUG MAGNETS

- Check for a repeated number after generating the next value.
- The digit-square helper must consume all digits with modulo and division.
- Do not treat reaching any small number as success; only 1 means happy.

#04 MiddleOfLinkedList

Easy

Runner midpoint Easy [Source](#) [LeetCode](#)**Before reading: why does slow stop exactly at the middle?****PROBLEM DESCRIPTION**

Given the head of a singly linked list, return the middle node. If there are two middle nodes, return the second one.

INPUT

head = [1, 2, 3, 4, 5, 6]

OUTPUT

node with value 4

TIME

$O(n)$

SPACE

$O(1)$

DATA STRUCTURES

Slow pointer, fast pointer, linked list nodes

WHEN TO RECOGNIZE IT

When you need the halfway node of a linked list without first counting every node.

VISUAL HOOK

slow +1

fast +2

fast ends

slow at middle

second middle

- 1 Move fast two nodes for every one node slow moves.
- 2 When fast cannot move further, it has consumed the list length.
- 3 Slow has moved half as many steps, so it points at the middle.

Imagine fast stretching the tape two nodes at a time while slow walks normally. When fast reaches the end, slow has covered exactly half the list.

CORE MOVE

Start slow and fast at head. Move slow one step and fast two steps while fast and fast.next exist. When the loop stops, slow is the required middle node.

CODE SKELETON

```
slow = head; fast = head
while fast != null and fast.next != null:
    slow = slow.next
    fast = fast.next.next
return slow
```

BRUTE FORCE / BASELINE

Count the list length first, then walk length / 2 nodes from head. It is simple but uses two passes instead of one.

ALTERNATE APPROACHES

- Store nodes in an array and return array[size / 2], but that costs $O(n)$ extra space.
- The slow/fast runner approach is the one-pass $O(1)$ space version.

BUG MAGNETS

- For even length, this loop returns the second middle because slow moves before fast becomes null.
- Check fast and fast.next before moving fast two nodes.
- Return the node itself, not just its value.

#05 PalindromicLinkedList

Easy

Fold and compare Easy [Source](#) [LeetCode](#)

Before reading: in reverse, which pointer saves the road ahead before you flip the arrow?

PROBLEM DESCRIPTION

Given the head of a singly linked list, return true if the node values read the same forward and backward, otherwise return false.

INPUT

head = [1, 2, 3, 5, 5, 3, 2, 1]

OUTPUT

true

TIME
 $O(n)$

SPACE
 $O(1)$

DATA STRUCTURES
Slow pointer, fast pointer, in-place linked list reversal

WHEN TO RECOGNIZE IT

When a linked list must be compared from both ends, but you cannot index backward like an array.

VISUAL HOOK

1 2 3 5 5 3 2 1

find middle

reverse right half

compare halves

post = next

next = prev

prev = current

current = post

1 vs 1

2 vs 2

3 vs 3

5 vs 5

- 1 Use slow and fast to find the middle.
- 2 Reverse the second half with four beats: save next in post, point current back to prev, move prev forward, then move current to post.
- 3 Compare head and reversed-half values one by one.

Fold the list at the middle. For the reverse, remember four beats: save the forward link, flip current backward, move prev onto current, then continue using the saved link.

CORE MOVE

Use slow and fast to reach the middle. Reverse from slow onward with the rhythm `post = current.next, current.next = prev, prev = current, current = post`. Then compare the reversed half with the original head until the reversed half ends.

CODE SKELETON

```
slow = middle using fast pointer
right = reverse(slow)
left = head
while right != null:
    if left.val != right.val: return false
    left = left.next
    right = right.next
return true

reverse(head):
    prev = null; current = head
    while current != null:
        post = current.next    // save road ahead
        current.next = prev    // flip arrow back
        prev = current         // grow reversed side
        current = post         // continue forward
    return prev
```

BRUTE FORCE / BASELINE

Copy all node values into an array, then use two indices from both ends. It is very simple, but costs $O(n)$ extra space.

ALTERNATE APPROACHES

- Use recursion to compare front and back during unwinding, but it still uses $O(n)$ call stack space.
- Restore the reversed half afterward if the caller expects the original list structure to remain unchanged.

BUG MAGNETS

- Always save `post = current.next` before `current.next = prev`; after the flip, current no longer knows the original next node.
- Return `prev` from `reverse`, because `temp/current` becomes null after the last node is processed.
- Compare only while the reversed second half is non-null.
- For odd length lists, starting reversal at `slow` includes the middle value, which still works because the compare stops on the shorter reversed walk.

#06 ReorderList

MediumSplit reverse weave Medium [Source](#) [LeetCode](#)

Before reading: which two next pointers must be saved before weaving one node from each half?

PROBLEM DESCRIPTION

Given a singly linked list, reorder it in-place from $L_0 \rightarrow L_1 \rightarrow \dots \rightarrow L_n$ into $L_0 \rightarrow L_n \rightarrow L_1 \rightarrow L_{n-1} \rightarrow L_2 \rightarrow L_{n-2}$ and so on.

INPUT

head = [1, 2, 3, 4, 5]

OUTPUT

[1, 5, 2, 4, 3]

TIME

$O(n)$

SPACE

$O(1)$

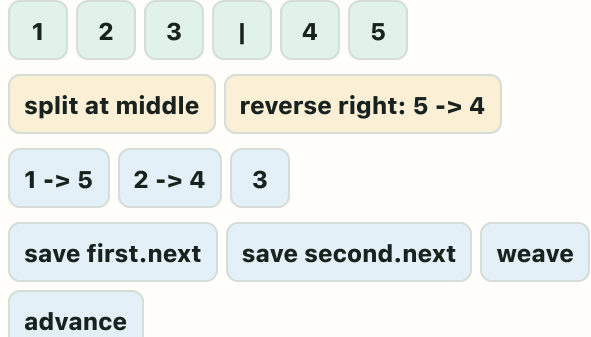
DATA STRUCTURES

Slow pointer, fast pointer, in-place linked list reversal, weaving pointers

WHEN TO RECOGNIZE IT

When a linked list must alternate nodes from the front and the back without using an array.

VISUAL HOOK



- 1 Find the middle, split the list, and reverse the second half.
- 2 Weave by storing first.next and second.next before reconnecting links.
- 3 Attach first -> second -> saved first-next, then advance both halves.

Think of it as folding a rope: cut after the middle, reverse the right half so the tail faces inward, then zip one node from the left and one node from the right.

CORE MOVE

Find the middle with slow and fast, detach the second half, reverse that half, then weave first and second by saving both next pointers before changing links.

CODE SKELETON

```
slow = node before second half
second = slow.next
slow.next = null
second = reverse(second)
first = head
while second != null:
    tempFirst = first.next
    tempSecond = second.next
    first.next = second
    second.next = tempFirst
    first = tempFirst
    second = tempSecond

reverse(head):
    prev = null; curr = head
    while curr != null:
        post = curr.next
        curr.next = prev
        prev = curr
        curr = post
    return prev
```

BRUTE FORCE / BASELINE

Copy all nodes into an array or deque and rebuild links from both ends. It is easier to visualize but costs $O(n)$ extra space.

ALTERNATE APPROACHES

- A stack can supply tail nodes while walking from the head, but it also uses $O(n)$ space and is messier to stop correctly.
- The split-reverse-weave approach keeps the required in-place $O(1)$ space behavior.

BUG MAGNETS

- Detach the first half with `slow.next = null` before weaving, otherwise old links can create a cycle.
- During weaving, save `tempFirst` and `tempSecond` before changing `first.next` and `second.next`.
- The middle-finding condition decides where the split happens; for odd lengths, the extra middle node stays at the end of the first half.

#07 RemoveNthNodeFromEndOfList

Medium

Fixed gap delete Medium [Source](#) [LeetCode](#)

Before reading: why does slow need to stop before the node being deleted?

PROBLEM DESCRIPTION

Given the head of a linked list and an integer n , remove the n th node from the end of the list and return the new head.

INPUT

head = [1, 2, 3, 4, 5], $n = 5$

OUTPUT

[2, 3, 4, 5]

TIME
 $O(n)$

SPACE
 $O(1)$

DATA STRUCTURES
Dummy node, slow pointer, fast pointer

WHEN TO RECOGNIZE IT

When you need to delete a node counted from the end in one pass, especially when the head itself might be removed.

VISUAL HOOK

- 1 Create a dummy before head so removing the first real node is normal.
- 2 Move fast n steps ahead to create a fixed gap.
- 3 Move both until fast is at the tail; slow is just before the node to remove.

Put a guard node before head. Push fast n steps ahead, then move slow and fast together. When fast reaches the last node, slow is parked just before the node to cut.

CORE MOVE

Create dummy $\rightarrow$ head. Move fast n steps from dummy. Move both pointers until fast.next is null. Delete slow.next and return dummy.next.

CODE SKELETON

```
dummy.next = head
slow = dummy; fast = dummy
repeat n times:
    fast = fast.next
while fast.next != null:
    slow = slow.next
    fast = fast.next
slow.next = slow.next.next
return dummy.next
```

BRUTE FORCE / BASELINE

Count the length first, compute the index from the front, then walk again to delete. It is clear but takes two passes.

ALTERNATE APPROACHES

- Use an array/list of node references and delete by index, but that costs $O(n)$ space.
- The dummy plus fixed-gap method is the clean one-pass $O(1)$ space version.

BUG MAGNETS

- The dummy node is what makes deleting the original head painless.
- Stop at `fast.next == null` so slow points to the node before the target.
- After deletion, return `dummy.next`, not the original head.

#08 CircularArrayLoop

Medium

Index runner cycle Medium [Source](#) [LeetCode](#)**Before reading: what two things make a detected cycle invalid here?****PROBLEM DESCRIPTION**

Given a circular array of non-zero jumps, return true if following jumps forms a cycle of length more than one where every jump moves in the same direction.

INPUT

```
nums = [2, -1, 1, 2, 2]
```

OUTPUT

```
true
```

TIME
 $O(n)$ **SPACE**
 $O(n)$ **DATA STRUCTURES**
Slow pointer, fast pointer, visited boolean array, circular index math**WHEN TO RECOGNIZE IT**

When each array index points to exactly one next index, so the array behaves like a linked list with circular wrapping.

VISUAL HOOK

0:+2 2:+1 3:+2 0

slow = next(i)

fast = next(next(i))

same sign only

no one-node loop

visited path

skip repeats

- 1 Pick a start direction and allow only indexes with that same sign.
- 2 Move slow by one jump and fast by two jumps using circular nextIndex.
- 3 If they meet, reject only the one-index self-loop; otherwise it is a valid cycle.

Treat every index like a node with one arrow. Slow takes one arrow, fast takes two arrows. The route only counts if every arrow keeps the same sign and the loop has more than one node.

CORE MOVE

For each unvisited index, remember its direction. Run slow and fast through nextIndex while both stay in that direction. If they meet, reject a self-loop, otherwise return true. Mark the explored same-direction path visited.

CODE SKELETON

```
for each unvisited start:
    direction = nums[start] > 0
    slow = start; fast = start
    while slow and fast stay same direction:
        nextFast = nextIndex(fast)
        if nextFast changes direction: break
        slow = nextIndex(slow)
        fast = nextIndex(nextFast)
        if slow == fast:
            if nextIndex(slow) == slow: break
            return true
    mark same-direction path visited
return false
```

BRUTE FORCE / BASELINE

From each index, follow jumps with a local set until repeat or invalid direction. It is easier to write but can revisit paths many times without global visited marking.

ALTERNATE APPROACHES

- You can mark exhausted paths in-place by setting values to 0, but this mutates the input.
- A global visited array keeps the input unchanged while preventing repeated exploration.

BUG MAGNETS

- Direction must be checked before every slow and fast move, including fast's intermediate next index.
- A self-loop where nextIndex(i) == i is invalid because cycle length must be greater than one.
- Use normalized modulo for negative jumps: if next < 0, add n.